

Review of the QA process.

Scrum:

In general we should put significant effort to improve ticket quality and the flow.

- ticket has clear structure and description (brief description on the top, expected measurable successful criteria like Acceptance criteria etc.)
- ticket type is used in a proper way. For a big portion of work we should use for instance Epic. Then for sub part of the business change we should use User Story, for poorly technical change we should use Task etc.
- The ticket should be proper sizing. Here I mean two things. First, which I saw specifically in Elisa, that descriptions are too long on User Story level. Then they should be splitted in smaller tickets. Also good practice is an assumption that the biggest ticket should be finished in a single sprint by one developer (usually two weeks).
- tickets readiness on each step. Put clear definition of 'Ready for development' status and encourage all stakeholders to participate. In other words, businesses answer all questions, designers provide final Figma, etc. before picking up the ticket from the developer. This good practice makes it a habit, especially for stakeholders outside the team.
- QA ticket assignment. Once a ticket is finished by the developer then it is formally assigned to the QA (currently it's Testing status in Jira). Then QA can decide if the ticket should be moved to "Ready to deploy" or back to the developer "In progress" state. This formality is important for better controlling the ticket flow, visibility. Also for managers better understanding of the current status of the work.
- sprint period, in case we decide to work in this approach. From my experience the best solution is to have 2 weeks of sprints. On one hand you have enough time to finish bigger tasks, on the other hand there is not too much work, which means that the whole team is aware of what we are going to obtain in the sprint. For bigger sprints 'team focus' is more challenging.
- perform regular demo sessions for bigger parts of the project. Could be less formal. Like we actually started to do this already.
- link PR's to the Jira ticket. This could be set as automatic action in the GitHub and it supports better tracking of the ticket, especially after some period of time, when we have to review a change from a business perspective. For the current task it's helpful, when we have more than one PR associated with the ticket.

QA process/activities:

- Mainly we have to work on improving ticket quality and proper use of them, Then in consequence the QA process will be kind of 'self-established'.
- What I definitely recommend is to finish development activities and then move the ticket to the 'Testing' phase and assign it to the QA. After testing and documenting test results, QA is moving to the next status 'Ready for deployment' etc.
- unit/integration tests for each repository,
- accessibility testing asap - I will provide code how to do that.

Test automation:

1. Tool selection.

- a. needs to be widely used on the market - better support, fixing issues etc.
 - b. decisions should be taken at least on the department level. Too many tools in the organisation for the same purpose is not effective, costly maintenance, costly learning curve etc.
 - c. I strongly recommend Playwright as a framework and then TypeScript as a language. Also to keep a high level of quality code of the framework, tools like ESLint and Sonarqube should be involved.
2. Test Environment.
- a. For e2e tests, the assumption is that tests run are stable and repeatable. Therefore we need a dedicated test environment, where tests could be performed.
3. Implementation of test automation.
- a. learning/discovery phase - use GitHub, however I recommend to run tests locally, until we will not have some level of confidence that tests are achieving their purpose and they are stable (e.g. could be required to add test id's as selectors in DOM). I guess, should take a couple of weeks.
 - b. setup standard process of running tests. Define when, how often they should be run. Run using an orchestration tool, like TeamCity and store results in a shareable location (on server).
 - c. part of the CI process, as optional steps. I mean is running, but not blocking the build.

4 SEO - open question